

Data Preprocessing Pipeline for IMU-Based Knee Joint Kinematics and Kinetics Prediction

Your Name

January 8, 2026

1 Introduction

This document provides a comprehensive description of the data preprocessing pipeline developed for predicting knee joint angles and moments from inertial measurement unit (IMU) sensor data in patients with knee osteoarthritis. The preprocessing pipeline consists of five sequential stages, each addressing specific data quality and formatting requirements necessary for deep learning model training. The complete pipeline transforms raw laboratory data files into normalized, time-standardized tensor representations suitable for convolutional neural network input.

The raw data originates from a motion capture laboratory where participants wore seven IMU sensors (bilateral shank, shoe, thigh positions plus one trunk sensor) while walking on an instrumented force plate. Each IMU recorded 9-axis data (3-axis accelerometer, 3-axis gyroscope, 3-axis magnetometer) at high sampling frequency, synchronized with ground reaction force measurements and optical motion capture-derived joint angles and moments. The ultimate goal is to train deep learning models that can predict the gold-standard optical motion capture joint biomechanics using only the wearable IMU sensor data, enabling clinical biomechanics assessment outside laboratory settings.

The preprocessing pipeline addresses several critical challenges inherent in real-world biomechanics data collection: inconsistent file naming conventions, variable-length time series due to natural gait variability, sensor failures and data dropouts, anatomical coordinate system differences between left and right legs, and the need for temporal normalization to enable batch processing in neural networks. Each stage of the pipeline was carefully designed to handle these challenges while preserving the essential biomechanical information contained in the sensor signals.

2 Stage 0: Data Organization and Format Standardization

2.1 Initial Data Structure and Challenges

The raw data arrived as text files with inconsistent naming conventions from multiple data collection sessions over several months. Each file contained sensor data for one or more gait cycles from a single walking trial, but the naming scheme varied between collection sessions and laboratory personnel. This inconsistency posed significant challenges for automated data processing, as file sorting algorithms would produce unpredictable orderings, and simple string matching operations

for patient identification would fail due to capitalization differences.

The initial filename format loosely followed this pattern:

```
p###_MMDDYY_speed_####_leg_imu_knee_angle_moment.txt
```

where **p** represented the patient identifier (sometimes lowercase 'p', sometimes uppercase 'P'), **MMDDYY** the visit date, **speed** the walking speed condition (walking, fast, slow), **####** a four-digit trial number, and **leg** indicated which leg's biomechanics were measured (either 'oaleg' for the osteoarthritis-affected leg or 'nonleg' for the contralateral leg). However, this format was not strictly enforced, leading to files with missing fields, extra underscores, or inconsistent abbreviations.

Beyond the naming issues, the files themselves were tab-delimited text files, which while human-readable, are not the preferred format for modern data science workflows. Tab-delimited files can have issues with inconsistent delimiters (mixing tabs and spaces), character encoding problems, and limited compatibility with certain data processing libraries. Converting these to comma-separated value (CSV) format provides better tool compatibility and more robust parsing.

2.2 Filename Standardization Procedure

The first critical step involved enforcing a strict, consistent naming convention across all data files. Inconsistent capitalization of patient identifiers was particularly problematic—a sorting algorithm might place 'p002' and 'P002' in different locations, and database queries searching for 'P002' would miss files labeled 'p002'. This could lead to incorrect train/test splits where the same patient appears in both sets, or missing data during analysis.

The standardization procedure systematically renamed all files to enforce uppercase patient identifiers:

Listing 1: Patient ID Standardization Implementation

```
for filename in natsorted(os.listdir(targetDir)):
    old_name = os.path.join(targetDir, filename)
    # Replace first character with uppercase 'P'
    new_name = os.path.join(targetDir, 'P' + filename[1:])
    os.rename(old_name, new_name)
```

The use of `natsorted()` is important here rather than Python's default `sorted()` function, because natural sorting correctly orders 'P2' before 'P10' (treating numbers as integers), whereas lexicographic sorting would place 'P10' before 'P2' (treating numbers as strings). This ensures files are processed in a logical, human-expected order.

After this standardization, all files strictly followed the format:

```
P###_MMDDYY_speed_####_leg_imu_knee_angle_moment.csv
```

This consistency enables reliable automated processing throughout the remainder of the pipeline, as regular expressions and string splitting operations can confidently extract specific fields from known positions.

2.3 Format Conversion: Text to CSV

The raw data files used tab characters as delimiters, which can be problematic in practice. Tab characters are often invisible in text editors, making it difficult to visually verify data structure. They can be inadvertently replaced with spaces during copy-paste operations. Different operating systems may handle tab characters differently, and some data processing tools have inconsistent tab-parsing implementations.

Converting to CSV format provides several advantages. CSV is a more universal standard with better tool support across programming languages and data analysis platforms. CSV files are more robust to whitespace inconsistencies, and most modern data science libraries optimize specifically for CSV parsing. The conversion was implemented using pandas' flexible file reading capabilities:

Listing 2: Format Conversion Implementation

```
for Name_datum in dataList:
    # Read tab-delimited text file
    read_file = pd.read_csv(os.path.join(txtDir, Name_datum),
                           sep='\t')
    # Write as comma-separated CSV
    read_file.to_csv(os.path.join(csvDir,
                                   Name_datum.replace(".txt", ".csv")))
```

The pandas library handles many edge cases automatically during this conversion, including proper escaping of commas within data fields, handling of missing values, and preservation of numeric precision. The resulting CSV files maintain identical data content while providing better compatibility with downstream processing tools.

2.4 Anatomical Side Information Integration

A critical piece of metadata for proper interpretation of the biomechanics data is the relationship between which leg was measured and which side has osteoarthritis. The filename field `leg` indicates whether the measured leg is 'oaleg' (osteoarthritis-affected) or 'nonleg' (contralateral), but this alone is insufficient. We also need to know whether the affected side is the patient's right or left leg, because this determines how coordinate systems must be transformed.

Consider a patient with right-leg osteoarthritis. If the file indicates 'oaleg' measurements, we know these come from the right leg. If it indicates 'nonleg', these come from the left leg. This information is essential for the coordinate system correction performed in Stage 1, where certain axes must be inverted to maintain consistent biomechanical interpretations across left and right legs.

This information was maintained in a separate demographics spreadsheet containing each patient's ID and affected side (Left/Right). The preprocessing pipeline merges this information directly into filenames:

Listing 3: Demographics Integration

```
# Load demographics file
list_demograph = pd.read_excel(dgDir, engine='openpyxl')

for Name_datum in dataList:
    # Extract patient ID from filename
```

```

patient_id = Name_datum.split('_')[0]

# Lookup affected side from demographics
sideInfo = list_demograph.loc[
    list_demograph['ID'] == patient_id, 'Side']

# Insert side information into filename
filename = Name_datum.replace(".txt", ".csv")
filename = '.'.join([
    filename.split('.')[0] + f'_{sideInfo.values[0]}',
    filename.split('.')[1]])

read_file.to_csv(os.path.join(csvDir, filename))

```

The final standardized filename format becomes:

P002_031220_w_0007_nonleg_R_imu_knee_angle_moment.csv

where the `R` or `L` suffix explicitly indicates the patient's affected side (Right or Left osteoarthritis). This embedded metadata eliminates the need to constantly reference external lookup tables during subsequent processing stages, reducing the risk of mismatched records and simplifying the codebase.

3 Stage 1: Data Quality Assessment and Step Segmentation

3.1 Understanding the Data Structure

Each CSV file contains time-series data with a highly specific column structure determined by the synchronized sensor array. The data structure reflects the hardware configuration used during data collection: seven IMU sensors, each with 9 channels (though ultimately only 6 are used), plus four channels of synchronized biomechanical reference data. Understanding this structure is critical for proper data extraction and validation.

For valid data collection, each complete gait step (from initial foot contact to subsequent contact of the same foot) requires exactly 75 columns of data:

IMU Sensor Data (63 columns total): The seven IMU sensors are positioned as follows: shank, shoe, and thigh on the non-operated leg (27 columns), shank, shoe, and thigh on the operated leg (27 columns), and one sensor on the trunk (9 columns). Each sensor provides 9 channels: 3-axis accelerometer (ACC_X, ACC_Y, ACC_Z measuring linear acceleration in m/s^2), 3-axis gyroscope (GYRO_X, GYRO_Y, GYRO_Z measuring angular velocity in rad/s), and 3-axis magnetometer (MAG_X, MAG_Y, MAG_Z measuring magnetic field strength). However, the magnetometer data was ultimately excluded from the final dataset due to environmental magnetic interference in laboratory settings, reducing each sensor's contribution from 9 to 6 channels in later stages.

Biomechanical Reference Data (12 columns total): The synchronized reference measurements include Ground Reaction Forces in three axes (GRF_X, GRF_Y, GRF_Z from force plate), Joint Angles in three planes (ANGLE_X for flexion/extension, ANGLE_Y for abduction/adduction, ANGLE_Z for internal/external rotation from optical motion capture), Joint Moments in three planes (MONM_X, MONM_Y, MONM_Z from inverse dynamics), and Normalized Moments

(MOBWHT_X, MOBWHT_Y, MOBWHT_Z, which are moments normalized by body weight and height for cross-subject comparison).

Some data files contain multiple consecutive gait cycles from a single walking trial. In these cases, the data is organized in an interleaved column format where all instances of each variable type are grouped together. If a file contains N gait steps, the total number of columns follows:

$$\text{Total Columns} = 2 + (75 \times N_{\text{steps}}) \quad (1)$$

where the first 2 columns are empty metadata columns added by the export process. This interleaved format means that for a two-step file, columns 2-76 contain step 1's data, while columns 77-151 contain step 2's data, but they're not simply sequential—rather, all instances of each sensor channel across both steps are grouped together.

3.2 Quality Control Criteria and Validation

Before any file can be included in the training dataset, it must pass strict quality control checks to ensure data integrity and completeness. These checks identify files with sensor failures, incomplete recordings, transmission errors, or other data quality issues that would corrupt model training.

The primary inclusion criterion checks the mathematical relationship between column count and the expected structure:

Listing 4: Structural Validation Logic

```
if (len(read_file.columns) - 2) % 75 == 0 and \
    (len(read_file.columns) - 2) // 75 > 0:
    MAXstepCount = (len(read_file.columns) - 2) // 75
    # File has valid structure, proceed with extraction
else:
    # File structure is invalid, move to exclusion folder
    read_file.to_csv(os.path.join(excludeDir, datum))
```

This validation accomplishes several things. The modulo check $(\text{len}(\text{read_file.columns}) - 2) \% 75 == 0$ ensures that after removing the 2 metadata columns, the remaining columns are an exact multiple of 75, indicating complete gait cycles with no partial data. The integer division $(\text{len}(\text{read_file.columns}) - 2) // 75$ calculates how many complete gait cycles are present. The condition > 0 ensures at least one complete cycle exists. Files failing these checks indicate incomplete data collection, early termination of trials, or corruption during file export or transfer.

A second critical quality check identifies sensor failures manifested as columns containing only zeros. When a wireless IMU sensor loses connection, experiences battery failure, or encounters transmission interference, the recording system typically fills the missing data with zeros rather than leaving gaps. These zero-filled columns provide no useful information and corrupt the data structure if not removed.

The zero-detection algorithm examines each column independently:

$$\text{Column is Valid} \iff \exists t \in [1, T] : x_t \neq 0 \quad (2)$$

where x_t represents the sensor value at time step t and T is the total number of time samples. In plain language, a column is valid if at least one time point contains a non-zero value.

Listing 5: Zero-Column Detection and Filtering

```
# Skip first 4 rows (headers) and convert to float
tmp = read_file.iloc[4:, :].astype(float)

# Keep only columns with at least one non-zero value
tmp = tmp.loc[:, (tmp != 0).any(axis=0)]

# Re-validate structure after removing zero columns
if (len(tmp.columns)) % 75 == 0 and \
    (len(tmp.columns)) // 75 > 0:
    # Structure still valid after filtering
    read_file.to_csv(os.path.join(includeDir, datum))
else:
    # Zero-column removal broke structure
    read_file.to_csv(os.path.join(excludeDir, datum))
```

The critical insight here is that after removing zero-columns, the file must still maintain the 75-column-per-step structure. If removing zero-columns leaves, for example, 148 columns (not divisible by 75), this indicates the sensor failure affected only some sensors during some steps, making the data unsuitable for our analysis which requires complete sensor arrays for all steps. Such files are excluded even though they contain some valid data, because mixing complete and incomplete steps would introduce inconsistency in the training data.

3.3 Step-by-Step Data Extraction Algorithm

Files containing multiple gait cycles must be de-interleaved and separated into individual step files. This is essential for the machine learning pipeline because each gait cycle should be an independent training sample. Keeping multiple steps in a single file would violate the independence assumption of stochastic gradient descent training.

The extraction algorithm must correctly parse the interleaved column format, which differs between IMU sensor data and biomechanical reference data due to how the data acquisition system organized the exports.

IMU Data Extraction Pattern:

For a file containing N total steps, where we want to extract step s (where $s \in [0, N - 1]$), the 63 IMU columns are accessed using:

$$\text{Column Index}_{\text{IMU}} = i \cdot N + s + 2, \quad i \in [0, 62] \quad (3)$$

where i iterates through each of the 63 sensor channels, N is the total number of steps, s is the specific step being extracted, and $+2$ accounts for the initial empty metadata columns. This formula works because the export process interleaved steps by sensor channel—all instances of the first sensor channel appear consecutively, followed by all instances of the second sensor channel, and so on.

Biomechanical Data Extraction Pattern:

The biomechanical reference data (12 columns per step) follows a different, more complex indexing pattern due to a different export organization:

$$\text{Column Index}_{\text{Biomech}} = (63 \cdot N) + j + (s \cdot 3) + 2 \quad (4)$$

where j ranges over $[0, 12N)$ but only indices where $j \bmod (3N) < 3$ are selected. This complex pattern reflects that the biomechanical data was organized in groups of three (X, Y, Z axes) with each group containing data for all steps before moving to the next biomechanical variable. The modulo condition extracts only the columns corresponding to the desired step while skipping other steps' data.

The complete extraction implementation brings these patterns together:

Listing 6: Multi-Step Data Extraction Implementation

```
for stepcount in range(0, MAXstepCount):
    # Extract IMU sensor data for this step
    DFperStep_imu = read_file.iloc[:,
        [idx * MAXstepCount + stepcount + 2
         for idx in range(0, 63)]]

    # Extract biomechanical reference data for this step
    DFperStep_SD = read_file.iloc[:,
        [(63 * MAXstepCount) + idx + stepcount * 3 + 2
         for idx in range(0, 12 * MAXstepCount)
         if idx % (3 * MAXstepCount) < 3]]

    # Combine IMU and biomechanical data horizontally
    DFperStep = pd.concat([DFperStep_imu, DFperStep_SD],
        axis=1).dropna(how='all')

    # Create filename indicating step number
    nametoSave = datum.split(".")[0] + "_" + \
        str(stepcount + 1) + "_Step.csv"

    # Save individual step file
    DFperStep.to_csv(os.path.join(includeDir, nametoSave))
```

The `dropna(how='all')` operation removes any rows that are completely empty, which can occur at the end of files where time series of different lengths were padded to equal length. The step numbering in filenames uses 1-indexing (`stepcount + 1`) for human readability, though internal processing uses 0-indexing.

This extraction creates individual files like:

P002_031220_w_0007_nonleg_R_1_Step.csv
P002_031220_w_0007_nonleg_R_2_Step.csv

each containing exactly 75 columns of data for a single gait cycle.

3.4 Coordinate System Correction for Left/Right Legs

A subtle but critical preprocessing step involves correcting coordinate system orientations based on which leg was measured. This is necessary because the IMU sensors' local coordinate systems are mirrored between left and right legs due to symmetric sensor placement, but the biomechanical interpretations must remain consistent.

Consider the shank sensor: when placed on the right leg, the sensor's local X-axis might point anteriorly (forward), but when the same sensor model is placed in the corresponding position on the left leg, its local X-axis points posteriorly (backward) due to the mirrored anatomy. If left uncorrected, a forward acceleration would appear positive for right leg data but negative for left leg data, even though the biomechanical event is identical.

The correction strategy depends on determining which leg corresponds to "nonleg" (the non-osteoarthritic leg). This is determined by cross-referencing the filename's `leg` field (`oaleg/nonleg`) with the side information (R/L) added in Stage 0:

If affected side is Right (`_R`):

- `oaleg` measurements → Right leg (no correction needed for reference frame)
- `nonleg` measurements → Left leg (requires axis inversion)

If affected side is Left (`_L`):

- `oaleg` measurements → Left leg (requires axis inversion)
- `nonleg` measurements → Right leg (no correction needed for reference frame)

The specific axes requiring inversion were determined through careful analysis of the sensor coordinate systems and biomechanical coordinate conventions. Not all axes require inversion—axes parallel to the anterior-posterior direction and the vertical direction typically need inversion for left legs, while the medial-lateral axis often does not, depending on the sensor mounting orientation.

Listing 7: Coordinate System Correction Implementation

```
# Extract metadata from filename
sideInfo = datum.split('_')[-3] # Get affected side (R/L)
legInfo = datum.split('_')[4]    # Get measured leg (oaleg/nonleg)

# Determine if nonleg is the left foot
nonleg_is_left = (sideInfo == 'R')

if nonleg_is_left:
    # Target columns requiring axis inversion for left leg
    # Excludes certain column ranges based on sensor configuration
    targetcol = [col for col in range(0, 27)
                  if not ((col % 9 > 0) and (col % 9 < 4)) or
                        ((col % 9 > 5) and (col % 9 < 9))]

    # Apply negative transformation
    for col in targetcol:
        read_file.iloc[4:, col] = read_file.iloc[4:, col] * -1
```

```

else:
    # oaleg is left foot, apply correction to those columns
    targetcol = [col + 27 for col in range(0, 27)
                  if not ((col % 9 > 0) and (col % 9 < 4)) or
                        ((col % 9 > 5) and (col % 9 < 9))]

    for col in targetcol:
        read_file.iloc[4:, col] = read_file.iloc[4:, col] * -1

```

The column selection logic `if not ((col % 9 > 0) and (col % 9 < 4))` identifies which axes within each 9-channel sensor group require inversion based on the sensor’s physical orientation. The modulo operation `col % 9` determines position within each sensor’s 9-channel block, allowing selective inversion of specific axes (e.g., `ACC_X` and `GYRO_X`) while preserving others (e.g., `ACC_Z` and `GYRO_Z`).

After this correction, all data is in a consistent biomechanical reference frame where positive/negative values have the same physiological meaning regardless of which anatomical leg was measured.

3.5 Processing Results and Data Flow Summary

The Stage 1 pipeline processes the 1,304 initial files through multiple quality gates, with files being excluded at each stage for different reasons. The final breakdown illustrates the stringency of quality control:

Initial Dataset: 1,304 raw data files from laboratory collection

After Structure Validation:

- 391 files excluded (incomplete gait cycles, incorrect column counts, corrupted exports)
- 913 files passed structural validation

Step Extraction Results:

- 828 single-step files → 828 individual gait cycles
- 85 two-step files → 170 individual gait cycles
- Total: 998 individual gait cycle samples

After Zero-Column Filtering:

- 95 files excluded (sensor failures, wireless dropouts)
- 903 files retained for further processing

Final Dataset: 903 validated, quality-controlled gait cycle samples, each containing 75 columns (63 IMU + 12 biomechanical) of time-series data, with corrected coordinate systems and consistent formatting.

This represents a retention rate of approximately 69% (903/1304), which is reasonable for real-

world biomechanics data collection where sensor failures, participant movement artifacts, and data transmission issues are common. The excluded files are carefully archived with documentation of exclusion reasons for potential future analysis or method development.

4 Stage 2: Visual Quality Inspection and Manual Curation

4.1 Motivation for Manual Review

While the automated quality control checks in Stage 1 successfully identify structural problems and obvious sensor failures, they cannot detect all data quality issues. Subtle problems such as drift in gyroscope baselines, periodic noise from electromagnetic interference, participant stumbles or gait abnormalities, loose sensor mounting causing movement artifacts, or calibration errors require expert visual inspection. These issues don't manifest as structural problems or zero-filled columns, but they can severely degrade model training by introducing non-physiological patterns that the network might learn instead of genuine biomechanical relationships.

Manual review by a trained researcher familiar with normal gait biomechanics is essential for identifying these nuanced quality issues. However, manually reviewing 903 files is time-consuming and error-prone without proper tooling. A custom Python-based review system was developed to streamline this process while maintaining accuracy and reproducibility.

4.2 Automated PDF Generation for Review

For each of the 903 data files, a multi-page PDF document was automatically generated showing all sensor channels and biomechanical variables plotted over the gait cycle. This visual representation allows rapid identification of anomalies that would be difficult to detect numerically.

Each PDF contains four pages of plots:

Page 1: Non-operated Leg IMU Sensors (9 subplots) Shows the 3-axis time series for each of the three sensors (shank, shoe, thigh) on the non-operated leg. Each subplot displays X, Y, and Z axes in different colors with separate y-axis scales to accommodate the different units and ranges of accelerometer versus gyroscope data.

Page 2: Operated Leg IMU Sensors (9 subplots) Mirrors Page 1's layout but for the osteoarthritis-affected leg sensors, enabling direct visual comparison between limbs to identify asymmetries.

Page 3: Trunk Sensor and Ground Reaction Forces (6 subplots) Displays trunk IMU data (3 subplots) plus ground reaction force components (3 subplots), which should show characteristic double-hump patterns during normal walking.

Page 4: Joint Kinematics and Kinetics (4 subplots) Shows the gold-standard optical motion capture joint angles, joint moments, and normalized moments in the three anatomical planes.

The plotting code uses a specialized multi-axis visualization approach to handle the different scales and units across sensor types:

Listing 8: Multi-Axis Plot Generation Function

```
def axis3plot(position, data, name):
```

```

"""Creates a 3-axis plot with separate y-axes for X, Y, Z"""
# Configuration for plot spacing
subAJ_right = 0.5      # Right margin for 3rd y-axis
subAJ_top = 1.5        # Top margin
subAJ_wspace = 0.5     # Width between subplots
sub_offset = 40        # Offset for 3rd y-axis

data = data.astype(float)

# Create main plot with custom axes
host = host_subplot(position, axes_class=AA.Axes)
plt.subplots_adjust(right=subAJ_right, top=subAJ_top,
                    wspace=subAJ_wspace)

# Create two additional y-axes
par1 = host.twinx()    # Second y-axis for Y data
par2 = host.twinx()    # Third y-axis for Z data

# Position third y-axis with offset
new_fixed_axis = par2.get_grid_helper().new_fixed_axis
par2.axis["right"] = new_fixed_axis(
    loc="right", axes=par2, offset=(sub_offset, 0))

par1.axis["right"].toggle(all=True)
par2.axis["right"].toggle(all=True)

# Plot three data series
host.set_xlabel(f"Time/{name}")
p1, = host.plot(range(len(data)), data.iloc[:, 0],
                label="X_axis")
p2, = par1.plot(range(len(data)), data.iloc[:, 1],
                label="Y_axis")
p3, = par2.plot(range(len(data)), data.iloc[:, 2],
                label="Z_axis")

host.legend()

# Color-code axes to match data lines
host.axis["left"].label.set_color(p1.get_color())
par1.axis["right"].label.set_color(p2.get_color())
par2.axis["right"].label.set_color(p3.get_color())

```

This multi-axis approach is crucial because accelerometer data (in m/s^2) and gyroscope data (in rad/s) have vastly different numerical ranges. Plotting them on the same y-axis would make one signal appear flat and unreadable. Similarly, the three axes of each sensor often have different dynamic ranges based on the sensor's orientation relative to gravity and motion direction.

The PDF generation loop processes all 903 files:

Listing 9: Batch PDF Generation

```

for datum in tqdm(dataList):
    read_file = pd.read_csv(os.path.join(rawDir, datum))

```

```

# Only generate if PDF doesn't already exist
if not os.path.exists(os.path.join(figDir,
                                   f'{datum.split(".")[0]}.pdf')):
    pp = PdfPages(os.path.join(figDir,
                               f'{datum.split(".")[0]}.pdf'))

    # Page 1: Non-operated leg sensors
    plt.figure(figsize=(15, 8))
    for i in range(0, 9):
        axis3plot(331 + i, read_file.iloc[4:, 3*i:3+i*3],
                  '_'.join(read_file.iloc[0, 3*i:3+i*3][0]
                           .split('_')[:-1]))

    plt.tight_layout()
    pp.savefig()
    plt.close()

# Pages 2-4 follow similar pattern for other data...
pp.close()

```

This PDF generation took approximately 2 hours for 903 files (about 8 seconds per file), with the bottleneck being file I/O over the network storage system rather than computation time. The generated PDFs serve as a permanent visual record of data quality at this processing stage.

4.3 Interactive Review Interface

With PDFs generated, a custom Python script using the `keyboard` library provides an efficient keyboard-driven review interface. This interface allows the reviewer to rapidly make decisions while automatically tracking all choices in a persistent Excel file.

The review system operates in several modes:

CHECK Mode (`-mode CHK`): Initial review of all un-reviewed files, progressing through files marked "TBD" (To Be Determined) in the tracking spreadsheet.

POSTPONED Mode (`-mode PP`): Re-review files marked as "PP" (Postponed) during initial review because the reviewer was uncertain about inclusion/exclusion.

EXPORT Mode (`-mode EXP`): After reviews are complete, automatically move files to appropriate directories based on review decisions.

The review workflow follows this logic:

Listing 10: Interactive Review System Core Logic

```

# Load or create tracking spreadsheet
listName = r"list_Excluded_byFig.xlsx"
if os.path.isfile(os.path.join(dataDir, listName)):
    list_Excluded_byFig = pd.read_excel(
        os.path.join(dataDir, listName), engine="openpyxl")
else:
    # Initialize new tracking file
    list_Excluded_byFig = pd.DataFrame(
        columns=["filename", "type", "note", "checker"])

```

```

list_Excluded_byFig["filename"] = dataList
list_Excluded_byFig["type"] = "TBD" # To Be Determined

# Get reviewer name for accountability
print("\n= Type your Name = ")
chk_name = input()

# Display current review status
print("\n== Status ==")
print(list_Excluded_byFig["type"].value_counts())

```

During CHECK mode review, the workflow for each file proceeds as follows:

Step 1: Initiate Review The reviewer presses 'w' to advance to the next un-reviewed file, or 'p' to terminate the session (which saves progress before exiting). This prevents accidental data loss if the reviewer needs to pause.

Step 2: Visual Inspection The system automatically opens the corresponding PDF file using the operating system's default PDF viewer. The reviewer examines all four pages of plots, looking for anomalies such as baseline drift indicating sensor calibration issues, spikes or discontinuities suggesting motion artifacts, asymmetric patterns between legs that don't match clinical expectations, ground reaction force curves that deviate from normal double-hump patterns, or missing data segments within otherwise valid files.

Step 3: Classification Decision Based on visual inspection, the reviewer makes one of three keyboard-based decisions:

- **Press 'q' (Include):** Data quality is acceptable for model training. The file is marked "IN" and the reviewer's name is recorded.
- **Press 'e' (Exclude):** Data contains quality issues that would corrupt model training. The file is marked "EXC" with reviewer attribution.
- **Press 's' (Postpone):** Data quality is ambiguous or requires second opinion. The system prompts for a text note explaining the concern, marks the file "PP", and moves to the next file.

Listing 11: Decision Capture and Recording

```

while True:
    if keyboard.read_key(suppress=True) == "q": # Include
        list_Excluded_byFig.loc[
            list_Excluded_byFig.filename == datum, "type"] = "IN"
        list_Excluded_byFig.loc[
            list_Excluded_byFig.filename == datum, "checker"] = chk_name
        print(f"Included!: {datum}")
        break

    if keyboard.read_key(suppress=True) == "e": # Exclude
        list_Excluded_byFig.loc[
            list_Excluded_byFig.filename == datum, "type"] = "EXC"
        list_Excluded_byFig.loc[

```

```

        list_Excluded_byFig.filename == datum, "checker"] = chk_name
    print(f"Excluded!: {datum}")
    break

if keyboard.read_key(suppress=True) == "s": # Postpone
    list_Excluded_byFig.loc[
        list_Excluded_byFig.filename == datum, "type"] = "PP"
    print("Enter your note:")
    reason = input()
    list_Excluded_byFig.loc[
        list_Excluded_byFig.filename == datum, "note"] = reason
    list_Excluded_byFig.loc[
        list_Excluded_byFig.filename == datum, "checker"] = chk_name
    print(f"Postponed!: {datum}")
    break

# Close PDF viewer
kill(plot.pid)

```

The `kill(plot.pid)` command forcefully closes the PDF viewer, preventing dozens of viewer windows from accumulating during the review session. The `kill()` function uses the `psutil` library to terminate both the main process and any child processes it may have spawned.

Step 4: Progress Saving After each decision, the tracking spreadsheet is updated in memory. When the reviewer presses 'p' to quit or when all files are reviewed, the spreadsheet is saved to disk:

Listing 12: Progress Persistence

```

print("\nwait for saving the list...")
list_Excluded_byFig.to_excel(
    os.path.join(dataDir, listName), index=False)
print("See you!")

```

This ensures that no review progress is lost, and the reviewer can resume from exactly where they stopped in a subsequent session.

4.4 Post-Review File Organization

After all files have been reviewed and postponed files resolved, EXPORT mode automatically organizes the dataset according to review decisions:

Listing 13: Automatic File Organization Based on Reviews

```

# Create destination directories
ensure_dir(includeDir)
ensure_dir(excludeDir)
ensure_dir(postpondDir)

for file in tqdm(list_Excluded_byFig["filename"]):
    # Copy files marked "IN" to include directory
    if (list_Excluded_byFig.loc[
        list_Excluded_byFig.filename == file, "type"] == "IN").bool():

```

```

        if not os.path.exists(os.path.join(includeDir, file)):
            shutil.copyfile(
                os.path.join(rawDir, file),
                os.path.join(includeDir, file))

    # Copy files marked "EXC" to exclude directory
    elif (list_Excluded_byFig.loc[
        list_Excluded_byFig.filename == file, "type"] == "EXC").bool():
        if not os.path.exists(os.path.join(excludeDir, file)):
            shutil.copyfile(
                os.path.join(rawDir, file),
                os.path.join(excludeDir, file))

    # Copy any remaining "PP" files to postponed directory
    elif (list_Excluded_byFig.loc[
        list_Excluded_byFig.filename == file, "type"] == "PP").bool():
        if not os.path.exists(os.path.join(postpondDir, file)):
            shutil.copyfile(
                os.path.join(rawDir, file),
                os.path.join(postpondDir, file))

```

This automated organization eliminates manual file moving errors and ensures perfect correspondence between the tracking spreadsheet and the actual file locations.

4.5 Review Outcomes and Quality Metrics

The manual review process identified additional quality issues not caught by automated checks. Common reasons for exclusion during visual review included:

Sensor Drift: Gradual baseline shifts in gyroscope readings indicating temperature-related drift or calibration decay over the trial duration. This appears as a slow upward or downward trend in the signal rather than oscillating around zero.

Motion Artifacts: Sharp spikes or discontinuities caused by sensor cable movement, loose mounting straps, or impacts with environmental objects. These artifacts are easily visible as sudden deviations from otherwise smooth trajectories.

Gait Abnormalities: Stumbles, hesitations, or compensatory movements that don't represent normal walking patterns. While these are genuine biomechanical events, they introduce non-steady-state gait patterns that could confuse the model.

Force Plate Issues: Ground reaction force curves that don't show the characteristic double-hump pattern (corresponding to heel strike and push-off phases), suggesting the participant didn't fully contact the force plate or stepped on the plate edge.

Incomplete Gait Cycles: Files where the time series starts or ends mid-cycle despite having the correct number of columns, indicating the trial was initiated or terminated at the wrong time.

The visual review phase represents a critical human-in-the-loop quality gate that complements automated validation. While time-consuming (approximately 10-15 hours total for 903 files), this investment significantly improves final dataset quality by eliminating subtle issues that would oth-

erwise corrupt model training.

5 Stage 3: Signal Filtering with Butterworth Low-Pass Filter

5.1 Rationale for Signal Filtering

Raw IMU sensor data contains high-frequency noise from multiple sources that must be removed before model training. Electronic noise originates from the IMU’s analog-to-digital converters and amplifier circuits, typically manifesting as random fluctuations at frequencies above the bandwidth of human movement (>15 Hz). Quantization noise arises from the discrete sampling of continuous signals, particularly noticeable in the least significant bits of the digital representation. Wireless transmission artifacts occur when electromagnetic interference or packet loss corrupts sensor readings. Environmental vibrations from floor impacts, clothing movement, and sensor mounting system resonances add high-frequency components unrelated to the biomechanics of interest.

Human walking involves movements primarily in the 0.5-5 Hz frequency range. The fundamental frequency of walking is the step frequency, typically 0.8-1.2 Hz for normal walking speed. Harmonics of this fundamental frequency extend to approximately 5-6 Hz for leg segment movements and up to 10-15 Hz for impact-related transients during heel strike. Frequencies above 15 Hz in IMU data almost exclusively represent noise rather than biomechanically meaningful information.

Retaining high-frequency noise during neural network training introduces several problems. The model may learn to associate noise patterns with outcomes rather than genuine biomechanical features, leading to poor generalization. High-frequency noise increases the dynamic range of input features, potentially causing numerical instability during gradient descent. Noise components increase the effective dimensionality of the input space, requiring more training data to achieve the same model performance. For these reasons, low-pass filtering is a standard preprocessing step in biomechanics signal processing.

5.2 Butterworth Filter Theory and Design

The Butterworth filter is a type of infinite impulse response (IIR) filter characterized by a maximally flat frequency response in the passband, meaning it has no ripples in the frequencies we want to preserve. This characteristic makes it particularly suitable for biomechanics applications where we want to avoid introducing artificial oscillations or distorting the shape of genuine movement signals.

Filter Transfer Function:

The magnitude response of an N -th order Butterworth low-pass filter is given by:

$$|H(f)|^2 = \frac{1}{1 + \left(\frac{f}{f_c}\right)^{2N}} \quad (5)$$

where f is the frequency, f_c is the cutoff frequency, and N is the filter order. The cutoff frequency f_c is defined as the frequency at which the magnitude response drops to $1/\sqrt{2}$ (approximately 0.707 or -3 dB) of its passband value.

Key Properties of Butterworth Filters:

Maximally Flat Passband: The first $2N - 1$ derivatives of $|H(f)|^2$ are zero at $f = 0$, ensuring the smoothest possible passband response. This means frequencies below the cutoff are transmitted with minimal distortion of their relative amplitudes.

Monotonic Response: The magnitude decreases monotonically from the passband through the stopband with no ripples, unlike Chebyshev filters which trade passband flatness for sharper rolloff but introduce amplitude ripples.

Rolloff Rate: The filter attenuates frequencies above cutoff at $20N$ dB per decade, where N is the filter order. A 4th-order filter provides 80 dB/decade rolloff, effectively eliminating high-frequency noise while maintaining computational efficiency.

Selecting Filter Parameters:

For this application, the filter design parameters were chosen as follows:

Cutoff Frequency ($f_c = 6$ Hz): This cutoff preserves all biomechanically relevant frequencies (0-5 Hz fundamental movements plus first-order harmonics up to 6 Hz) while beginning to attenuate higher-frequency noise. The choice of 6 Hz rather than a higher value like 10 Hz provides a safety margin below the typical noise floor (>15 Hz) while ensuring that the transition band (6-15 Hz) adequately separates signal from noise.

Filter Order ($N = 4$): Fourth-order Butterworth filters provide an excellent balance between sharp frequency selectivity (80 dB/decade rolloff) and computational stability. Lower orders ($N=2$) have gentler rolloff and may not adequately attenuate noise. Higher orders ($N=6$ or greater) provide sharper rolloff but increase computational cost and can introduce numerical stability issues, particularly near the filter's stability boundaries. Fourth-order is a standard choice in biomechanics signal processing literature.

Sampling Frequency ($f_s = 100$ Hz): The IMU sensors recorded at 100 Hz, which satisfies the Nyquist criterion ($f_s > 2f_{\max}$) with substantial margin. For a cutoff of 6 Hz, the normalized cutoff frequency used in digital filter design is:

$$f_n = \frac{f_c}{f_s/2} = \frac{6}{50} = 0.12 \quad (6)$$

This normalized frequency is used when designing the discrete-time filter from its continuous-time prototype.

5.3 Digital Filter Implementation

The Butterworth filter is implemented using SciPy's signal processing tools, which provide numerically stable algorithms for IIR filter design and application:

Listing 14: Butterworth Filter Design and Application

```
from scipy import signal

# Design 4th-order Butterworth low-pass filter
# cutoff=6 Hz, sampling rate=100 Hz
```

```

b, a = signal.butter(N=4, Wn=6, btype='low',
                    analog=False, fs=100)

# Apply filter to each column of sensor data
for col in range(0, len(datum.columns)):
    if col < 63: # Only filter IMU sensor columns
        datum.iloc[:, col] = signal.filtfilt(
            b, a, datum.iloc[:, col].astype(float))

```

The `signal.butter()` function returns the filter coefficients in two arrays: b contains the numerator coefficients (feedforward terms) and a contains the denominator coefficients (feedback terms) of the discrete transfer function:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \dots + b_N z^{-N}}{a_0 + a_1 z^{-1} + a_2 z^{-2} + \dots + a_M z^{-M}} \quad (7)$$

where z^{-1} represents a unit delay operator in the discrete-time domain.

Zero-Phase Filtering with `filtfilt`:

A critical implementation detail is the use of `signal.filtfilt()` rather than `signal.lfilter()`. Standard IIR filters introduce phase shifts—different frequency components are delayed by different amounts as they pass through the filter. This phase distortion can artificially shift the timing relationships between sensor channels, which is particularly problematic for biomechanics where the relative timing of movements is biomechanically meaningful.

The `filtfilt()` function implements zero-phase filtering using a forward-backward filtering technique:

1. Apply the filter forward in time: $y_1[n] = H(z) \cdot x[n]$
2. Reverse the filtered signal: $y_1[-n]$
3. Apply the filter again in the reversed direction: $y_2[n] = H(z) \cdot y_1[-n]$
4. Reverse again to restore original time order: $y[n] = y_2[-n]$

Mathematically, this achieves a frequency response of:

$$|H_{\text{zero-phase}}(f)|^2 = |H(f)|^4 \quad (8)$$

The phase response becomes:

$$\phi_{\text{zero-phase}}(f) = \phi(f) - \phi(f) = 0 \quad (9)$$

because the phase shift introduced during forward filtering is exactly canceled by the phase shift during backward filtering (which has opposite sign).

This zero-phase property preserves the temporal alignment of all sensor channels and maintains the precise timing of biomechanical events like heel strike and toe-off. The trade-off is that the effective filter order doubles (4th-order becomes equivalent to 8th-order magnitude response), providing even sharper frequency selectivity with 160 dB/decade rolloff.

Boundary Effects and Padding:

The `filtfilt()` function internally applies padding to the signal boundaries using the `padtype='odd'` or `padtype='even'` options. This padding mitigates the startup transients that occur when an IIR filter begins processing from an arbitrary initial condition. Without padding, the first and last few samples of the filtered signal can exhibit artificial distortions.

The padding works by reflecting the signal about its endpoints and extrapolating using odd or even symmetry, creating a smooth transition that minimizes edge effects. For a signal of length N , typical padding extends the signal by $3 \times \max(\text{len}(a), \text{len}(b))$ samples on each end before filtering, then truncates back to the original length afterward.

5.4 Selective Filtering Strategy

A crucial detail in the implementation is that **only the 63 IMU sensor columns are filtered**, while the 12 biomechanical reference columns (ground reaction forces, joint angles, joint moments) are left unfiltered:

Listing 15: Selective Column Filtering

```
for col in range(0, len(datum.columns)):
    if col < 63: # First 63 columns are IMU sensors
        datum.iloc[:, col] = signal.filtfilt(
            b, a, datum.iloc[:, col].astype(float))
    # Columns 63-74 (biomechanical data) remain unfiltered
```

This selective filtering reflects the different signal characteristics and purposes of these data types. The IMU sensors are the model inputs and contain high-frequency noise that must be removed for effective training. The biomechanical variables (angles and moments) are the model outputs and were already optimally filtered during the optical motion capture post-processing using different algorithms appropriate for kinematic data. Applying a second round of filtering to the outputs would unnecessarily smooth the target variables, potentially reducing the model's ability to learn fine details of the biomechanical patterns.

Additionally, the ground reaction force data from the force plate has different frequency content than IMU data—impact transients during heel strike contain important high-frequency components that characterize loading patterns. A 6 Hz low-pass filter would remove these transients, eliminating valuable information about impact characteristics.

5.5 Filtering Implementation and Batch Processing

The complete filtering pipeline processes all quality-controlled files from Stage 2:

Listing 16: Batch Filtering Implementation

```
from scipy import signal
```

```

# Set up paths
rawDir = r'...\Included_checked\RAW_AXIS_corrected'
filteredDir = r'...\Included_checked\FILT'
ensure_dir(filteredDir)

# Design filter once (reused for all files)
b, a = signal.butter(N=4, Wn=6, btype='low',
                    analog=False, fs=100)

# Get list of files to filter
dataList = natsorted([f for f in os.listdir(rawDir)
                      if f.endswith('.csv')])

for file in tqdm(dataList):
    # Load data
    datum = pd.read_csv(os.path.join(rawDir, file))

    # Filter only IMU columns (first 63 columns)
    for col in range(0, min(63, len(datum.columns))):
        datum.iloc[:, col] = signal.filtfilt(
            b, a, datum.iloc[:, col].astype(float))

    # Save filtered data
    nameForFilt = "F_" + file # Add "F_" prefix
    datum.to_csv(os.path.join(filteredDir, nameForFilt),
                 index=False)

```

The filtered files are saved with an "F_" prefix to clearly distinguish them from raw data, maintaining traceability throughout the pipeline. This naming convention allows easy identification of processing stage from filename alone.

5.6 Filter Validation and Quality Assurance

To validate the filtering implementation, frequency domain analysis was performed on representative signals before and after filtering:

Power Spectral Density Analysis:

For a random sample of accelerometer and gyroscope channels, the power spectral density (PSD) was computed using Welch's method:

$$\text{PSD}(f) = \frac{1}{T} |\mathcal{F}\{x(t)\}|^2 \quad (10)$$

where $\mathcal{F}$ denotes the Fourier transform and T is the signal duration.

The PSD plots confirmed that:

- Frequencies below 6 Hz retain their original power (i 0.5 dB attenuation)
- The -3 dB point occurs at exactly 6 Hz as designed

- Frequencies above 15 Hz are attenuated by ≥ 40 dB (99% power reduction)
- No spurious peaks or artifacts are introduced by the filtering

Time-Domain Validation:

Visual inspection of filtered signals confirmed that:

- Gait cycle patterns remain clearly visible and recognizable
- Characteristic features (heel strike transients, swing phase trajectories) are preserved
- High-frequency jitter is eliminated while signal smoothness is maintained
- No phase shifts are visible between related sensor channels (e.g., shank accelerometer X-axis leads thigh accelerometer X-axis by the same time delay before and after filtering)

The successful validation confirms that the Butterworth filtering effectively removes noise while preserving all biomechanically relevant information in the signals.

6 Stage 4: Time Normalization via Resampling

6.1 The Variable-Length Time Series Problem

After filtering, each gait cycle file contains high-quality sensor data, but a critical problem remains: **the time series have variable lengths**. Natural human gait exhibits inter-cycle and inter-subject variability in temporal characteristics. Different participants walk at different speeds, taking more or fewer samples per gait cycle. Even the same participant across multiple trials shows cycle-to-cycle variations in cadence and stride time. Individual gait cycles in the dataset range from approximately 80 to 200 time samples, reflecting this natural variability.

This variable-length structure is fundamentally incompatible with standard neural network architectures for several reasons:

Batch Processing Requirement: Modern deep learning relies on mini-batch stochastic gradient descent, where multiple training samples are processed simultaneously in a batch to leverage GPU parallelism. A batch is represented as a tensor with shape (batch_size, time_steps, features). All samples in a batch must have identical dimensions—mixing 80-sample and 150-sample sequences in the same batch is mathematically undefined.

Fixed Architecture Dimensions: Convolutional and fully-connected layers have fixed weight matrix dimensions determined at network initialization. A conv1d layer expects input of shape (batch, channels, length). If length varies between samples, the layer cannot consistently apply the same learned filters. While recurrent networks (RNNs, LSTMs) can technically handle variable-length sequences, they are computationally expensive and less suitable for the temporal pattern recognition task at hand compared to CNNs.

Consistent Temporal Resolution: For the network to learn meaningful temporal patterns (e.g., "the knee flexion peak occurs at 15% of the gait cycle"), all samples must represent time in the same coordinate system. Without normalization, sample 50 in an 80-sample sequence (62.5% through

the cycle) represents a different phase of gait than sample 50 in a 150-sample sequence (33.3% through the cycle).

The standard solution in biomechanics is ****time normalization****: resampling all gait cycles to a fixed number of time points representing percentage of gait cycle completion. The convention is 101 time points (0% to 100% in 1% increments), allowing each time point to represent a specific phase of the gait cycle across all subjects and trials.

6.2 Resampling Theory and Methods

Time normalization requires resampling the signal from its original sampling rate (variable number of samples per cycle) to a fixed number of samples (101 points). This is fundamentally different from the low-pass filtering in Stage 3—filtering changes frequency content while preserving time base, whereas resampling changes the time base while preserving frequency content (within the limits of the new Nyquist frequency).

Mathematical Formulation:

Given a discrete signal $x[n]$ with N_{original} samples, we want to generate a new signal $y[m]$ with exactly 101 samples, where $m \in [0, 100]$. Each output sample $y[m]$ corresponds to a non-integer position in the original signal:

$$t_m = m \cdot \frac{N_{\text{original}} - 1}{100} \quad (11)$$

Since t_m is generally not an integer, we must interpolate between adjacent original samples to estimate $y[m]$.

Two primary resampling methods were considered:

Method 1: FFT-Based Resampling (`signal.resample`)

This method performs resampling in the frequency domain using the Fourier transform:

$$X[k] = \mathcal{F}\{x[n]\} = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N} \quad (12)$$

$$y[m] = \mathcal{F}^{-1}\{X[k]\}_{\text{at 101 points}} \quad (13)$$

The algorithm computes the FFT of the original signal, then computes the inverse FFT with a different number of output points. If resampling to more points (upsampling), the frequency spectrum is zero-padded. If resampling to fewer points (downsampling), the high-frequency components are truncated.

Advantages: Theoretically optimal for band-limited signals satisfying the Nyquist criterion. No assumptions about signal smoothness or continuity. Computationally efficient using FFT algorithms.

Disadvantages: Assumes periodic boundaries—treats the signal as if it wraps around, with the last sample adjacent to the first. This can introduce discontinuities at boundaries if the signal doesn't naturally return to its starting value. Can exhibit ringing artifacts (Gibbs phenomenon) near sharp transitions due to truncation of the Fourier series.

Method 2: Polyphase Filtering (`signal.resample_poly`)

This method performs resampling using polyphase filtering, which is essentially interpolation followed by decimation using FIR filters:

1. Upsample by factor L (insert $L - 1$ zeros between samples)
2. Apply low-pass FIR filter to interpolate intermediate values
3. Downsample by factor M (keep every M -th sample)

The resampling ratio is $\frac{L}{M} = \frac{101}{N_{\text{original}}}$, and the algorithm automatically determines appropriate integer factors using continued fractions approximation.

Advantages: Handles non-periodic signals gracefully using boundary padding. The padding type can be specified (e.g., `padtype='line'` extends the signal linearly at boundaries). Generally produces smoother results with fewer artifacts for biomechanical signals which often have continuous derivatives.

Disadvantages: Slightly more computational cost than FFT method. The low-pass filter design requires careful selection of parameters.

6.3 Implementation and Method Selection

Both methods were initially implemented and compared on test signals:

Listing 17: FFT-Based Resampling Implementation

```
def resample_ALL(datum):
    """Resample all columns to exactly 101 points using FFT"""
    newColumns = datum.columns
    norm_datum = pd.DataFrame(columns=newColumns)

    for i in range(len(datum.columns)):
        raw_Data = datum.iloc[:, i]
        # FFT-based resampling to 101 points
        norm_Data = signal.resample(raw_Data, 101)
        norm_datum[datum.columns[i]] = norm_Data

    return norm_datum
```

Listing 18: Polyphase Filtering Implementation

```
def resample_poly_ALL(datum):
    """Resample all columns to 101 points using polyphase filtering"""
    newColumns = datum.columns
    norm_datum = pd.DataFrame(columns=newColumns)
```

```

for i in range(len(datum.columns)):
    raw_Data = datum.iloc[:, i]
    # Polyphase resampling with linear boundary padding
    norm_Data = signal.resample_poly(
        raw_Data, 101, len(raw_Data), padtype='line')
    norm_datum[datum.columns[i]] = norm_Data

return norm_datum

```

Visual comparison of the two methods on representative gait cycles revealed that:

FFT Method Issues: Boundary artifacts were visible, particularly for signals that don't naturally form closed loops (e.g., trunk accelerations that have different values at heel strike versus subsequent heel strike). The assumption of periodicity created artificial discontinuities visible as small spikes at the 0% and 100% time points.

Polyphase Method Performance: The linear padding (`padtype='line'`) effectively eliminated boundary artifacts by smoothly extending the signal trends at endpoints. The resulting signals appeared visually smoother and more consistent with the original signal characteristics.

Based on this comparison, **the polyphase filtering method was selected for final implementation**, as documented in the code comment:

```

# filt_datum = resample_ALL(datum) # Basic FFT method (commented out)
filt_datum = resample_poly_ALL(datum) # Use polyphase - better results

```

6.4 Batch Processing and Quality Control

The time normalization is applied to all filtered files from Stage 3:

Listing 19: Time Normalization Batch Processing

```

from scipy import signal

# Set up directories
filteredDir = r'...\FILT'
normalizedDir = r'...\NORM'
ensure_dir(normalizedDir)

# Get files to process
dataList = natsorted([f for f in os.listdir(filteredDir)
                      if f.endswith('.csv')])

for file in tqdm(dataList):
    # Load filtered data
    datum = pd.read_csv(os.path.join(filteredDir, file))

    # Apply polyphase resampling to 101 time points
    norm_datum = resample_poly_ALL(datum)

    # Save normalized data with "N_" prefix
    nameForNorm = "N_" + file
    norm_datum.to_csv(os.path.join(normalizedDir, nameForNorm),

```

```
index=False)
```

The "N_" prefix indicates normalized data, so files now have the naming pattern:

N_F_P002_031220_w_0007_nonleg_R_1_Step.csv

where "N" indicates normalized time, "F" indicates filtered, and the remainder preserves the original metadata.

Post-Resampling Validation:

After resampling, verification confirms that:

- All files contain exactly 101 rows (time points)
- Column structure (75 columns) is preserved
- Characteristic gait patterns are recognizable in plots
- No introduction of NaN or infinite values
- Temporal relationships between related channels are maintained
enditemize

6.5 Implications for Neural Network Training

The time normalization to 101 points has several important implications for the neural network architecture and training process:

Consistent Input Dimensions: All training samples now have shape (42, 101) after removing magnetometer channels in later processing. This allows the use of efficient convolutional architectures that can learn temporal patterns at specific phases of the gait cycle. For example, a convolutional filter centered at time point 15 will consistently examine the early stance phase across all samples.

Interpretability: Each time index directly corresponds to a percentage of gait cycle completion. If the model learns important features at time points 15-25, these correspond to the 15-25% phase of gait (loading response through mid-stance), which has clear biomechanical interpretation. This temporal correspondence enables meaningful visualization of learned features and model attention patterns.

Computational Efficiency: Fixed-length sequences enable efficient batched operations on GPUs. A batch of 32 samples forms a tensor of shape (32, 42, 101), which can be processed through convolutional layers in a single highly parallelized operation. Variable-length sequences would require either padding (wasting computation on padded values) or sequential processing (losing parallelism benefits).

Temporal Invariance vs. Phase-Specific Learning: The normalization makes the model phase-specific rather than temporally invariant. The model learns that "knee flexion angle peaks at time point 60" rather than "knee flexion peaks at some unspecified time." This is

appropriate for gait analysis where events occur at stereotyped phases of the gait cycle, unlike tasks like speech recognition where events can occur at arbitrary times.

However, the normalization also introduces a subtle consideration: participants who naturally walk with different cadences are now represented identically in time-normalized space. A fast walker’s knee flexion velocity in the original time base differs from a slow walker’s, but after normalization both show similar angular velocities per percentage of cycle. This is generally desirable for gait analysis, as we’re interested in the pattern of movement through the gait cycle rather than absolute timing, but it means the model cannot learn cadence-related features.

6.6 Preservation of Dynamic Information

An important question is whether resampling distorts the dynamic information in velocity and acceleration signals. Consider a participant who takes 120 samples for one gait cycle (original) versus 101 samples after resampling:

Original sampling rate: If the original signal was collected at 100 Hz and contains 120 samples, the gait cycle duration was 1.2 seconds.

After resampling: The signal is represented by 101 samples, but these no longer correspond to a specific sampling frequency in real time. Instead, they represent 101 equally-spaced samples through the gait cycle.

Velocity and acceleration interpretation: The resampling preserves the pattern of velocity and acceleration ****with respect to gait cycle percentage**** rather than real time. This is exactly what we want for biomechanics analysis, where we care about the pattern of movement through the gait cycle phase rather than absolute timing.

For signals representing rates of change (gyroscopes measure angular velocity), the resampling effectively rescales the amplitude based on the original cycle duration. A participant who completes the same movement pattern in less time will show higher velocity signals in the original data, but after normalization, both participants show similar velocity patterns through the normalized cycle. This rescaling is handled automatically by the resampling algorithm and doesn’t require explicit correction.

7 Stage 5: Dataset Assembly and Final Formatting

7.1 Magnetometer Channel Removal

After time normalization, the data files contain 75 columns (63 IMU + 12 biomechanical), with IMU data including magnetometer channels. However, magnetometer data was excluded from the final training dataset due to several practical considerations that emerged during exploratory analysis.

Environmental Interference: Magnetometers measure Earth’s magnetic field and use it as a reference for absolute orientation. However, indoor laboratory environments contain substantial magnetic interference from structural steel, electrical wiring, electronic equipment, and metal force plates. This interference creates spatially varying magnetic fields that

distort the measurements, making them unreliable for orientation estimation in the specific experimental setting.

Redundant Information: For the task of predicting joint angles and moments from sensor data during straight-line walking, absolute spatial orientation (which magnetometers provide) is less critical than relative motion patterns (which accelerometers and gyroscopes capture). The accelerometer and gyroscope data contain sufficient information about limb segment kinematics without requiring absolute compass heading.

Generalization Concerns: Magnetometer readings are highly environment-specific. A model trained on data including magnetometer information from one laboratory setting would not generalize well to other environments with different magnetic signatures. Excluding magnetometers improves model portability to real-world clinical and home settings.

The magnetometer removal reduces the IMU channel count from 63 to 42 (7 sensors $\times$ 6 channels: 3 accelerometer + 3 gyroscope). The function `makeColumnsWOMAG()` generates the appropriate column headers:

Listing 20: Column Header Generation Without Magnetometer

```
def makeColumnsWOMAG():
    """Creates column names excluding magnetometer channels"""
    SIDEIDX = ['non', 'oa'] # Non-operated vs operated leg
    PARTIDX = ['shank', 'shoe', 'thigh'] # Body segments
    TYPEIDX = ['ACC', 'GYRO', 'MAG'] # Sensor types
    AXISIDX = ['X', 'Y', 'Z'] # Coordinate axes

    # Generate 54 columns for leg sensors (27 per leg)
    COL_imu_legs = [
        f'{SIDEIDX[int(i//(54/2))]}_',
        f'{PARTIDX[(i//(len(TYPEIDX)*len(AXISIDX))%len(PARTIDX))]}_',
        f'{TYPEIDX[(i//(len(AXISIDX))%len(TYPEIDX))]}_',
        f'{AXISIDX[i%len(AXISIDX)]}'
        for i in range(0, 54)
    ]

    # Generate 9 columns for trunk sensor
    Col_imu_trunk = [
        f'trunk_{TYPEIDX[(i//(len(AXISIDX))%len(TYPEIDX))]}_',
        f'{AXISIDX[i%len(AXISIDX)]}'
        for i in range(0, 9)
    ]

    # Generate 12 columns for biomechanical data
    FPTYPEIDX = ['GRF', 'ANGLE', 'MONM', 'MOBWHT']
    Col_FP = [
        f'{FPTYPEIDX[(i//(len(AXISIDX))%len(FPTYPEIDX))]}_',
        f'{AXISIDX[i%len(AXISIDX)]}'
        for i in range(0, 12)
    ]

    # Combine all columns
    newColumns = COL_imu_legs + Col_imu_trunk + Col_FP
```

```

# Filter out magnetometer columns
newColumnsWOMAG = [col for col in newColumns
                    if 'MAG' not in col]

return newColumnsWOMAG

```

This function generates standardized column names like:

```

non_shank_ACC_X, non_shank_ACC_Y, non_shank_ACC_Z,
non_shank_GYRO_X, non_shank_GYRO_Y, non_shank_GYRO_Z,
...
trunk_ACC_X, trunk_ACC_Y, trunk_ACC_Z,
trunk_GYRO_X, trunk_GYRO_Y, trunk_GYRO_Z,
GRF_X, GRF_Y, GRF_Z,
ANGLE_X, ANGLE_Y, ANGLE_Z,
...

```

The final data structure has:

- **Input features (42 columns):**
 - * Non-operated leg: shank (6), shoe (6), thigh (6) = 18 channels
 - * Operated leg: shank (6), shoe (6), thigh (6) = 18 channels
 - * Trunk: 6 channels
- **Output targets (3 columns):** ANGLE_X, ANGLE_Y, ANGLE_Z OR MOBWHT_X, MOBWHT_Y, MOBWHT_Z (depending on prediction task)

7.2 Data Standardization and Normalization Strategy

Before neural network training, the data must be scaled to appropriate numerical ranges. Different scaling strategies serve different purposes in machine learning pipelines:

Min-Max Scaling (0-1 Range):

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (14)$$

This scaling maps all values to the $[0, 1]$ interval. The advantage is bounded output range which can be beneficial for neural networks with sigmoid or tanh activation functions. The disadvantage is sensitivity to outliers—a single extreme value can compress the majority of the data into a narrow range.

Standardization (Z-score Normalization):

$$x_{\text{standardized}} = \frac{x - \mu}{\sigma} \quad (15)$$

where μ is the mean and σ is the standard deviation. This centers the data at zero with unit variance. It's less sensitive to outliers than min-max scaling and is preferred for many neural network architectures, particularly those using batch normalization layers.

Robust Scaling (Median and IQR):

$$x_{\text{robust}} = \frac{x - \text{median}(x)}{\text{IQR}(x)} \quad (16)$$

where IQR is the interquartile range (75th percentile - 25th percentile). This is most robust to outliers but can be less stable when the data distribution has heavy tails.

For this project, ****Min-Max scaling to [0, 1] was selected**** for several reasons:

- The final neural network architecture uses ReLU activations (output range [0,)) and outputs representing bounded physical quantities (joint angles and moments have natural physical ranges)
- Min-Max scaling preserves the absolute relationships between positive and negative values, which is important for signals like acceleration that meaningfully cross zero
- The bounded output range helps prevent numerical instability during early training iterations
- The data has already been cleaned through multiple quality control stages, reducing outlier concerns

7.3 Critical Principle: Fit Scalers Only on Training Data

A fundamental principle in machine learning is preventing ****data leakage**** from the test set into the training process. Data leakage occurs when information from the test set influences model training in any way, leading to overly optimistic performance estimates that don't reflect real-world generalization.

If scalars are fit on the entire dataset (training + testing combined), the scaling parameters ($x_{\min}$, $x_{\max}$) are computed using test set values. This leaks information about the test set's statistical properties into the model training process. The model implicitly has access to information about the range of test set values, which wouldn't be available in real-world deployment.

The correct procedure is:

1. **Split data first:** Divide into training and testing sets based on participant IDs (ensuring no participant appears in both sets)
2. **Fit scalars on training data only:** Compute $x_{\min}$ and $x_{\max}$ using only training samples
3. **Transform training data:** Apply the fitted scaler to training data

4. **Transform test data:** Apply the same scaler (with training-derived parameters) to test data

This procedure ensures that test data remains completely unseen during scaler fitting, preventing any form of information leakage. Test samples may end up with scaled values outside $[0, 1]$ if they exceed the training set's range, which is acceptable and expected.

7.4 K-Fold Cross-Validation Data Partitioning

To maximize data utilization while maintaining rigorous evaluation, the dataset employs **5-fold cross-validation** at the participant level. This approach provides robust performance estimates while ensuring each data sample is used for both training and testing exactly once across all folds.

Participant-Level Splitting:

The critical requirement is that all gait cycles from a single participant must appear together in either the training or testing set for a given fold. Splitting individual gait cycles from the same participant across training and testing would create data leakage because gait patterns are highly individual—knowing how participant P005 walks in the training set provides information about how P005 will walk in the test set.

The splitting process uses scikit-learn's `KFold` with participant IDs:

Listing 21: Participant-Level K-Fold Splitting

```
from sklearn.model_selection import KFold

# Get unique participant IDs
arrName = listFromxlsx.patientID.unique() # 44 unique participants

# Initialize 5-fold cross-validation
kfold = KFold(n_splits=5, random_state=41, shuffle=True)

# Create folds based on participant IDs
for fold_idx, (train_pids, test_pids) in enumerate(kfold.split(arrName)):
    # train_pids and test_pids are indices into arrName
    train_participants = arrName[train_pids] # ~35 participants
    test_participants = arrName[test_pids]    # ~9 participants

    # Convert participant IDs to data file indices
    train_indices, test_indices = kfold2subfold(
        arrName, listFromxlsx, train_pids, test_pids)
```

The `kfold2subfold()` function maps participant indices to data file indices:

Listing 22: Participant-to-File Index Mapping

```
def kfold2subfold(arrName, listData, train, test):
    """
    Convert participant-based splits to file-based indices.
```

```

Args:
    arrName: Array of participant IDs
    listData: DataFrame with 'patientID' column
    train: Indices of training participants
    test: Indices of testing participants

Returns:
    arrTrain: List of training file indices
    arrTest: List of testing file indices
"""
arrTrain = []
arrTest = []

# Collect all file indices for training participants
for pID in arrName[train]:
    # Find all rows where patientID matches
    idxofID = listData.index[listData['patientID'] == pID]
    arrTrain.extend(idxofID.to_list())

# Collect all file indices for testing participants
for pID in arrName[test]:
    idxofID = listData.index[listData['patientID'] == pID]
    arrTest.extend(idxofID.to_list())

return arrTrain, arrTest

```

This produces a typical split like:

- **Fold 0:** 35 training participants (722 gait cycles), 9 test participants (154 gait cycles)
- **Fold 1:** 35 training participants (667 gait cycles), 9 test participants (209 gait cycles)
- **Fold 2:** 35 training participants (722 gait cycles), 9 test participants (154 gait cycles)
- **Fold 3:** 35 training participants (670 gait cycles), 9 test participants (206 gait cycles)
- **Fold 4:** 36 training participants (723 gait cycles), 8 test participants (153 gait cycles)

The slight variations in split sizes reflect different numbers of gait cycles per participant.

7.5 Scaler Fitting and Data Transformation

For each fold, separate scalers are created and fitted on the training data only:

Listing 23: Scaler Fitting Per Fold

```

from sklearn.preprocessing import MinMaxScaler
from pickle import dump

for fold in range(5):
    # Load training data for this fold (after all previous processing)
    train_data_X = load_normalized_filtered_data(train_indices)
    train_data_Y_angle = load_angle_targets(train_indices)
    train_data_Y_moment = load_moment_targets(train_indices)

```

```

# Create separate scalers for inputs and outputs
scaler4X = MinMaxScaler() # For IMU input features
scaler4Y_angle = MinMaxScaler() # For angle targets
scaler4Y_moment = MinMaxScaler() # For moment targets

# Fit scalers ONLY on training data
scaler4X.fit(train_data_X)
scaler4Y_angle.fit(train_data_Y_angle)
scaler4Y_moment.fit(train_data_Y_moment)

# Transform training data
scaled_X_train = scaler4X.transform(train_data_X)
scaled_Y_angle_train = scaler4Y_angle.transform(train_data_Y_angle)
scaled_Y_moment_train = scaler4Y_moment.transform(
    train_data_Y_moment)

# Transform test data using TRAINING-derived scalers
test_data_X = load_normalized_filtered_data(test_indices)
test_data_Y_angle = load_angle_targets(test_indices)
test_data_Y_moment = load_moment_targets(test_indices)

scaled_X_test = scaler4X.transform(test_data_X)
scaled_Y_angle_test = scaler4Y_angle.transform(test_data_Y_angle)
scaled_Y_moment_test = scaler4Y_moment.transform(
    test_data_Y_moment)

# Save fitted scalers for later use (inverse transform predictions)
dump(scaler4X, open(f"{fold}_fold_scaler4X.pkl", 'wb'))
dump(scaler4Y_angle, open(f"{fold}_fold_scaler4Y_angle.pkl", 'wb'))
dump(scaler4Y_moment, open(f"{fold}_fold_scaler4Y_moment.pkl", 'wb'))

```

The fitted scalers are saved as pickle files because they're needed during model evaluation to transform predictions back to physical units (inverse transform). Without the scalers, predicted values in $[0, 1]$ space cannot be compared to real-world joint angles in degrees or moments in Nm.

7.6 Tensor Reshaping for CNN Input

The scaled data must be reshaped into the specific tensor format expected by the convolutional neural network. The target format is:

$$\mathbf{X}_{\text{train}} : (N_{\text{train}}, 42, 101, 1) \quad (17)$$

where:

- N_{train} is the number of training samples

- 42 is the number of input feature channels (IMU sensors)
- 101 is the number of time steps (normalized gait cycle percentage)
- 1 is a singleton dimension often required by certain frameworks

Similarly for outputs:

$$\mathbf{Y}_{\text{train}} : (N_{\text{train}}, 3, 101, 1) \quad (18)$$

where 3 represents the X, Y, Z axes of joint angles or moments.

The reshaping process involves careful consideration of memory layout:

Listing 24: Data Reshaping for Neural Network Input

```
# After scaling, data is in format: (N_samples * 101, 42)
# where 101 consecutive rows represent one gait cycle

X_train = []
Y_angle_train = []
Y_moment_train = []

for i in range(len(train_indices)):
    # Extract rows for this sample (101 consecutive rows)
    sample_X = scaled_X_train[i*101 : (i+1)*101, :] # Shape: (101,
    42)
    sample_Y_angle = scaled_Y_angle_train[i*101 : (i+1)*101, :] #
    (101, 3)
    sample_Y_moment = scaled_Y_moment_train[i*101 : (i+1)*101, :] #
    (101, 3)

    # Reshape using Fortran-order (column-major) flattening
    # This preserves the time-series structure properly
    X_train.append(sample_X.flatten('F').reshape(-1, 1))
    Y_angle_train.append(sample_Y_angle.flatten('F').reshape(-1, 1))
    Y_moment_train.append(sample_Y_moment.flatten('F').reshape(-1, 1))

# Convert lists to numpy arrays
final_X_train = np.array(X_train) # Shape: (N_train, 4242, 1)
final_Y_angle_train = np.array(Y_angle_train) # Shape: (N_train, 303,
1)
final_Y_moment_train = np.array(Y_moment_train) # Shape: (N_train,
303, 1)
```

The use of `.flatten('F')` is crucial—'F' specifies Fortran-order (column-major) flattening, which reads down columns first before moving to the next column. This preserves the temporal structure where consecutive elements in the flattened array represent consecutive time points for the same feature channel.

The final shapes are:

- Input: $(N, 4242, 1)$ where $4242 = 42 \times 101$ (42 channels $\times$ 101 time points)

- Output angles: $(N, 303, 1)$ where $303 = 3 \times 101$ (3 axes $\times$ 101 time points)
- Output moments: $(N, 303, 1)$ (same structure as angles)

7.7 Final Dataset Persistence

The complete preprocessed dataset is saved in compressed NumPy format for efficient loading during model training:

Listing 25: Dataset Persistence

```
from numpy import savez_compressed

for fold in range(5):
    # Save training data
    savez_compressed(
        f"{fold}_fold_final_train.npz",
        final_X_train=final_X_train,
        final_Y_angle_train=final_Y_angle_train,
        final_Y_moBWHT_train=final_Y_moment_train,
        participant_train=participant_ids_train
    )

    # Save testing data
    savez_compressed(
        f"{fold}_fold_final_test.npz",
        final_X_test=final_X_test,
        final_Y_angle_test=final_Y_angle_test,
        final_Y_moBWHT_test=final_Y_moment_test,
        participant_test=participant_ids_test
    )
```

The `savez_compressed()` function uses gzip compression, typically reducing file sizes by 60-80% compared to uncompressed storage. The participant ID arrays are included to enable participant-level analysis of results.

Loading the preprocessed data during training is straightforward:

Listing 26: Loading Preprocessed Dataset

```
# Load data for a specific fold
data = np.load(f"{fold}_fold_final_train.npz")
X_train = data['final_X_train']
Y_angle_train = data['final_Y_angle_train']
Y_moment_train = data['final_Y_moBWHT_train']
participant_ids = data['participant_train']
```

8 Preprocessing Pipeline Summary

The complete preprocessing pipeline transforms raw laboratory data files into training-ready neural network inputs through five carefully designed stages:

Stage 0 (Organization): Standardizes filenames, converts formats, and embeds anatomical metadata. Ensures consistency and traceability throughout the pipeline.

Stage 1 (Quality Control): Validates data structure, extracts individual gait cycles, removes sensor failures, and corrects coordinate systems. Reduces 1,304 raw files to 903 validated gait cycles.

Stage 2 (Manual Curation): Visual inspection identifies subtle quality issues through automated PDF generation and keyboard-driven review interface. Maintains high data quality standards.

Stage 3 (Signal Filtering): Fourth-order Butterworth low-pass filter (6 Hz cutoff) removes high-frequency noise while preserving biomechanical signals. Zero-phase filtering maintains temporal relationships between sensor channels.

Stage 4 (Time Normalization): Polyphase resampling transforms variable-length sequences to fixed 101-point representations. Enables batch processing and creates phase-normalized inputs for CNN training.

Stage 5 (Dataset Assembly): Removes magnetometers, applies Min-Max scaling with careful train/test separation, implements 5-fold cross-validation at participant level, and creates properly formatted tensors for neural network consumption.

The final dataset consists of:

- 903 high-quality gait cycles from 44 participants
- 5-fold cross-validation splits maintaining participant independence
- Input tensors: $(N, 4242, 1)$ representing 42 IMU channels $\times$ 101 time points
- Output tensors: $(N, 303, 1)$ representing 3 axes $\times$ 101 time points
- Preserved scalars enabling inverse transformation to physical units

This preprocessing pipeline embodies best practices in biomechanics signal processing and machine learning data preparation, with careful attention to data quality, proper train/test separation, and reproducibility. Each stage builds upon the previous, creating a robust foundation for neural network training and evaluation.